

EDA

EDA – 1: Bike Details Dataset

1. What is the range of selling prices in the dataset?

The range of selling prices tells us the difference between the highest selling price and the lowest selling price in the dataset. To find it, we first find the minimum and maximum values in the `selling_price` column and calculate $\text{Range} = \text{Maximum selling price} - \text{Minimum selling price}$. In Python, we can use `df['selling_price'].min()` to find the minimum and `df['selling_price'].max()` to find the maximum. Then we subtract the minimum from the maximum. This gives us an idea of how widely the prices of the used bikes vary in the dataset.

2. What is the median selling price for bikes in the dataset?

The median selling price is the middle value when all bike selling prices are arranged from smallest to largest. If there is an odd number of observations, the middle value is the median; if there is an even number, the median is calculated using the average of the two middle values. In Python, we can simply use `df['selling_price'].median()`. Median is particularly useful in EDA because it is less affected by extremely expensive or extremely cheap bikes than the mean. Therefore, it can give us a better idea of the typical selling price when outliers are present.

3. What is the most common seller type?

The seller type tells us whether the bike was listed by an Individual or a Dealer. To find the most common seller type, we count how many records belong to each category and select the category with the highest count. In Python, `df['seller_type'].value_counts()` gives the number of bikes listed by each seller type. The category with the largest count is the most common seller type. This analysis helps us understand whether used-bike listings in this dataset are mainly coming from individual sellers or dealers. The dataset specifically identifies seller type as either Individual or Dealer.

4. How many bikes have driven more than 50,000 kilometers?

This question asks us to count the bikes where the value of `km_driven` is greater than 50,000. We can filter the DataFrame using a condition such as `df[df['km_driven'] > 50000]` and then use `len()` or `.shape[0]` to count the records. For example, `len(df[df['km_driven'] > 50000])` gives the number of bikes that have been driven more

than 50,000 km. This analysis helps us understand how many bikes in the dataset have relatively high usage.

5. What is the average km_driven value for each ownership type?

This question asks us to calculate the average kilometers driven separately for each ownership category, such as 1st owner, 2nd owner, etc. We can use Pandas `groupby()` for this purpose. For example, `df.groupby('owner')['km_driven'].mean()` groups the bikes according to their ownership type and calculates the average kilometers driven for each group. This allows us to compare the usage of bikes based on their ownership history and understand whether bikes with different ownership levels have different average driving distances.

6. What proportion of bikes are from the year 2015 or older?

This question asks us to determine what percentage or proportion of bikes were manufactured in 2015 or earlier. First, we filter the rows where `year <= 2015`, count those bikes, and divide the result by the total number of bikes. The formula is $\text{Proportion} = \text{Number of bikes from 2015 or older} \div \text{Total number of bikes}$. If we want the answer as a percentage, we multiply the proportion by 100. This tells us what fraction of the dataset consists of older bikes.

7. What is the trend of missing values across the dataset?

Missing-value analysis is an important part of EDA because missing information can affect the results of our analysis and Machine Learning models. We can check missing values in every column using `df.isnull().sum()`. This gives the number of missing values in each feature. We can also calculate percentages using `df.isnull().sum() / len(df) * 100`. By examining these values, we can identify which columns have missing data and decide whether to remove the records, fill the missing values, or use another suitable method. In simple words, this question asks where missing data exists and how much of it exists.

8. What is the highest ex_showroom_price recorded, and for which bike?

The `ex_showroom_price` represents the original showroom price of the bike. To answer this question, we find the maximum value in the `ex_showroom_price` column and then identify the corresponding bike name from the `name` column. In Pandas, we can use `df.loc[df['ex_showroom_price'].idxmax()]`. The `idxmax()` function finds the row containing the highest value, and `loc` retrieves that complete row. This gives us both the highest showroom price and the bike associated with it.

9. What is the total number of bikes listed by each seller type?

This question is similar to finding the most common seller type, but here we need the count for every seller category. We can use `df['seller_type'].value_counts()`. The result

will show how many bikes were listed by Individual sellers and how many were listed by Dealers. This is useful because it gives us a simple comparison of the contribution of each seller type to the dataset.

10. What is the relationship between `selling_price` and `km_driven` for first-owner bikes?

This question asks us to study whether the selling price changes when the kilometers driven changes, but only for bikes that are first-owner bikes. First, we filter the dataset for the first-owner category. Then we can calculate the correlation between `selling_price` and `km_driven` or create a scatter plot. A scatter plot is especially useful because it allows us to visually see whether prices generally decrease as kilometers driven increase. If the points show a downward pattern, it indicates a negative relationship. However, correlation does not prove that kilometers driven alone causes the change in price.

11. Identify and remove outliers in the `km_driven` column using the IQR method.

An outlier is a value that is unusually far from the majority of the observations. The IQR, or Interquartile Range, method is commonly used to identify such values. First, we calculate the first quartile, $Q1$, and third quartile, $Q3$. Then we calculate $IQR = Q3 - Q1$. The lower boundary is $Q1 - 1.5 \times IQR$, and the upper boundary is $Q3 + 1.5 \times IQR$. Values outside these boundaries are considered outliers. We can then filter the DataFrame to keep only values between the lower and upper boundaries. This technique is useful because extremely high `km_driven` values could influence statistical analysis and Machine Learning models.

12. Perform a bivariate analysis to visualize the relationship between year and `selling_price`.

Bivariate analysis means studying the relationship between two variables. Here, the two variables are year and `selling_price`. A scatter plot can be used to visualize this relationship, with manufacturing year on the X-axis and selling price on the Y-axis. We can use `plt.scatter(df['year'], df['selling_price'])`. By looking at the plot, we can understand whether newer bikes generally have higher selling prices than older bikes. This is useful for identifying trends in bike depreciation and resale value.

13. What is the average depreciation in selling price based on the bike's age (current year - manufacturing year)?

Depreciation means the reduction in an asset's value over time. To study depreciation, we first calculate the bike's age using $\text{Bike Age} = \text{Current Year} - \text{Manufacturing Year}$. Then we can group the bikes according to their age and calculate the average selling price for each age group. For example, we can use `df['age'] = current_year - df['year']` and then `df.groupby('age')['selling_price'].mean()`. Comparing selling prices across

different ages helps us understand how the resale value changes as a bike becomes older.

14. Which bike names are priced significantly above the average price for their manufacturing year?

This question asks us to compare each bike's selling price with the average selling price of bikes manufactured in the same year. First, we calculate the average selling price for each manufacturing year using `groupby('year')['selling_price'].transform('mean')`. Then we compare every bike's actual selling price with the average price of its year. Bikes whose prices are considerably higher than their year's average can be identified as unusually expensive. This can help us identify premium models, popular brands, or bikes whose condition may justify a higher resale price.

15. Develop a correlation matrix for numeric columns and visualize it using a heatmap.

A correlation matrix shows the correlation between every pair of numerical variables in the dataset. For the bike dataset, numerical columns include values such as `selling_price`, `year`, `km_driven`, and `ex_showroom_price`. We can calculate the matrix using `df.corr(numeric_only=True)`. To visualize it, we can use a heatmap, which represents different correlation values using different shades. Values close to +1 indicate strong positive relationships, values close to -1 indicate strong negative relationships, and values near 0 indicate weak linear relationships. The heatmap makes it easier to quickly identify important relationships between numerical features. The PDF specifically asks for a correlation matrix and heatmap as the final Bike EDA task.

EDA – 2: Car Sales Dataset

1. What is the average selling price of cars for each dealer, and how does it compare across different dealers?

This question asks us to calculate the **average car selling price separately for every dealer**. We can use `groupby()` on the dealer column and calculate the mean of the price column. For example, `df.groupby('Dealer_Name')['Price ($)'].mean()` gives the average price for each dealer. We can then compare these averages to determine which dealers generally sell more expensive or cheaper cars. A bar chart can also be used to make the comparison easier to understand visually.

2. Which car brand (Company) has the highest variation in prices, and what does this tell us about the pricing trends?

Here, **variation** means how widely the prices differ within each car company. We can calculate a measure such as standard deviation for the Price (\$) column after grouping by Company. For example, `df.groupby('Company')['Price ($)'].std()` gives the standard deviation of prices for each company. The company with the highest standard deviation has the greatest variation in its prices. This may indicate that the company offers cars across a wider range of models, features, or price categories, although variation alone does not tell us the exact reason.

3. What is the distribution of car prices for each transmission type, and how do the interquartile ranges compare?

This question compares car prices between **Manual and Automatic** transmission types. We can use a box plot with transmission type on one axis and price on the other. A box plot shows the distribution, median, and quartiles of the data. The **Interquartile Range (IQR)** is calculated as $Q3 - Q1$ and represents the range containing the middle 50% of prices. Comparing the IQRs tells us which transmission category has greater variation in its middle 50% of prices. The dataset identifies transmission as Manual or Automatic.

4. What is the distribution of car prices across different regions?

This question asks us to compare car prices among different **dealer regions**, such as North, South, etc. We can group the data according to Dealer_Region and examine the distribution of Price (\$) for each region. A box plot is useful because it shows the median, spread, and possible outliers for every region. This analysis can help us understand whether cars in some regions tend to have higher or lower selling prices.

5. What is the distribution of cars based on body styles?

Body style describes the type or shape of the car, such as Sedan or SUV. To find the distribution, we count the number of cars belonging to each body style using `value_counts()`. A bar chart or countplot can then be used to visualize these counts. This tells us which body styles are most common in the dataset and provides an understanding of the composition of the used-car market.

6. How does the average selling price of cars vary by customer gender and annual income?

This question asks us to study car prices in relation to **customer gender and annual income**. We can group the data by gender and calculate average price, or create income groups and compare the average prices between them. For example, income can be divided into Low, Medium, and High brackets and then the average selling price can be calculated for each group. This allows us to explore whether differences in customer income or gender are associated with differences in the cars purchased.

7. What is the distribution of car prices by region, and how does the number of cars sold vary by region?

This question combines **price distribution and sales count**. First, we can use a box plot to compare car prices across regions. Then we can use `value_counts()` or `groupby()` to count how many cars were sold in each region. This gives us two pieces of information: how much cars cost in different regions and how many cars were sold in each region. The dataset includes `Dealer_Region` as the geographical region of the dealer.

8. How does the average car price differ between cars with different engine sizes?

The Engine feature represents the engine type, such as **V6 or I4**. We can group the dataset by engine type and calculate the average Price (\$) for each group. For example, `df.groupby('Engine')['Price ($)'].mean()` gives the average price for each engine category. Comparing these values helps us understand whether certain engine types are associated with higher or lower average selling prices.

9. How do car prices vary based on the customer's annual income bracket?

Instead of examining every individual income value, we can divide customers into **income brackets**, such as low, medium, and high income. We then calculate the average or distribution of car prices within each bracket. A box plot or bar chart can be used to compare the groups. This analysis helps us explore whether customers with different income levels tend to purchase cars at different price ranges.

10. What are the top 5 car models with the highest number of sales, and how does their price distribution look?

First, we count the number of sales for every car model using `value_counts()` on the `Model` column and select the top five models. Then we filter the dataset to those five models and study their price distributions using a box plot. This allows us to identify the most frequently sold models and also see whether their selling prices are generally low, high, or widely spread.

11. How does car price vary with engine size across different car colors, and which colors have the highest price variation?

This question combines three variables: **engine type, car color, and price**. We can group or visualize prices based on engine type and color and use box plots or grouped charts to compare them. To identify colors with the highest price variation, we can calculate the standard deviation of price for each color. The color with the largest standard deviation has the greatest variation in prices within this dataset. This type of analysis is called **multivariate analysis** because more than two variables are being examined together.

12. Is there any seasonal trend in car sales based on the date of sale?

A **seasonal trend** means a repeated pattern in sales during particular months or periods. Since the dataset contains a `Date` column representing the date of the transaction, we can convert it into a proper date format and extract the month using something such as `pd.to_datetime(df['Date']).dt.month`. Then we can count the number of sales for each month and visualize the results using a line chart or bar chart. If some months consistently have more sales than others, that may indicate a seasonal pattern.

13. How does the car price distribution change when considering different combinations of body style and transmission type?

Here we examine **two categorical variables together—body style and transmission—and compare their relationship with price**. For example, we can compare Sedan-Manual, Sedan-Automatic, SUV-Manual, and SUV-Automatic groups. A box plot with combinations of body style and transmission can show the price distribution for each group. This helps us understand whether transmission type affects prices differently for different body styles.

14. What is the correlation between car price, engine size, and annual income of customers, and how do these features interact?

This question asks us to examine the relationship between numerical variables such as **car price, engine-related numerical information if available, and annual income**. We can calculate a correlation matrix and visualize it using a heatmap. A positive correlation means that two variables tend to increase together, while a negative correlation means they tend to move in opposite directions. However, if engine

information is stored as categories such as V6 and I4, we cannot directly calculate a normal numerical correlation from those labels without an appropriate transformation.

15. How does the average car price vary across different car models and engine types?

This question combines **Model, Engine, and Price**. We can group the data by both Model and Engine and calculate the average selling price. For example, `df.groupby(['Model', 'Engine'])['Price ($)'].mean()` can provide the average price for each model-engine combination. This allows us to compare whether different engine types within the same model are associated with different average selling prices.

EDA – 3: Amazon Sales Data

1. What is the average rating for each product category?

This question asks us to calculate the **mean rating separately for every product category**. We can use `groupby()` with the category column and calculate the mean of the rating column. For example, `df.groupby('category')['rating'].mean()` provides the average rating for each category. Comparing these values allows us to determine which categories have higher or lower average customer ratings.

2. What are the top rating_count products by category?

`rating_count` represents the **number of people who voted for the Amazon rating**. A higher rating count means that more customers have provided ratings for that product. To answer this question, we can group products by category and identify products with the highest `rating_count` within each category. Sorting the values in descending order makes it easier to find the products that have received the greatest number of ratings.

3. What is the distribution of discounted prices vs. actual prices?

This question compares two numerical variables: **actual price and discounted price**. We can use a scatter plot where actual price is on one axis and discounted price is on the other. We can also use histograms or box plots to examine their individual distributions. Since discounted price represents the price after discount and actual price represents the original price, we would generally expect discounted prices to be lower than or equal to actual prices in valid records.

4. How does the average discount percentage vary across categories?

This question asks us to calculate the **average discount percentage for each product category**. We can use `df.groupby('category')['discount_percentage'].mean()`. After calculating the averages, we can use a bar chart to compare categories. This helps us understand which categories generally offer larger or smaller discounts.

5. What are the most popular product names?

To identify popular products, we need to define popularity using a measurable feature available in the dataset. A useful measure is `rating_count`, because it represents how many people voted for the product's rating. We can sort products by `rating_count` in descending order and identify products with the highest counts. These products can be considered the most popular according to customer rating participation.

6. What are the most popular product keywords?

A **product keyword** is an important word or term appearing in product names or descriptions. To find popular keywords, we would need to process text from columns such as `product_name` or `about_product`, remove unnecessary words such as common stop words, and count the frequency of the remaining words. Words appearing most frequently can then be considered common product keywords. This is a basic example of **text analysis in EDA**.

7. What are the most popular product reviews?

The dataset contains review-related fields including `review_title` and `review_content`. To identify popular reviews, we need a measurable definition of "popular," such as reviews associated with products having high rating counts or another available review-engagement measure. The provided dataset description does not specify a separate review-like or helpfulness score, so the exact meaning of "popular reviews" would need to be defined before calculating it. This is important in EDA because we should not assume a measurement that is not present in the data.

8. What is the correlation between discounted_price and rating?

This question asks whether **discounted price and product rating have a linear relationship**. We can calculate it using Pandas, for example `df['discounted_price'].corr(df['rating'])`. The result lies between -1 and +1. A positive value indicates that higher discounted prices tend to be associated with higher ratings, a negative value indicates an opposite tendency, and a value close to zero indicates little linear relationship. Remember that **correlation does not prove causation**.

9. What are the Top 5 categories based on the highest ratings?

To find the top five categories, we first calculate the average rating for each category using `groupby()` and `mean()`. We then sort the results from highest to lowest and select the first five categories. This gives us the categories with the highest average customer ratings rather than simply the products with the highest individual ratings.

10. Identify any potential areas for improvement or optimization based on the data analysis.

This question asks us to use the results of the previous EDA tasks to identify useful improvements. For example, if some categories have low average ratings, they may need product-quality improvement; if some products have high rating counts but relatively low ratings, their customer satisfaction may need attention; and if certain categories have unusually low discounts compared with others, pricing strategies could be reviewed. However, the exact recommendations should be based on the actual results obtained from the dataset rather than assumptions. The purpose is to use EDA findings to identify **patterns, problems, and opportunities for improvement**.

EDA – 4: Spotify Dataset

1. Read the dataframe, check null value if present then do the needful, check duplicate row, if present then do the needful.

The first step in EDA is to understand and clean the dataset. We can read the CSV file using Pandas, for example `df = pd.read_csv('file.csv')`. To check missing values, we use `df.isnull().sum()`, which tells us how many null values exist in each column. If missing values are present, we decide whether to fill them or remove the corresponding records depending on the situation. To check duplicate rows, we can use `df.duplicated().sum()`. If duplicates exist and they represent repeated records, we can remove them using `df.drop_duplicates()`. This step ensures that the dataset is cleaner before performing further analysis.

2. What is the distribution of popularity among the tracks in the dataset? Visualize it using a histogram.

A **distribution** tells us how the popularity values are spread across the tracks. A histogram divides popularity values into ranges called bins and shows how many tracks fall into each range. We can use `plt.hist(df['Popularity'])` to create the histogram. By examining the histogram, we can determine whether most tracks have low, medium, or high popularity and whether the distribution is concentrated or spread out. The PDF specifically asks for a histogram to visualize this distribution.

3. Is there any relationship between the popularity and the duration of tracks? Explore this using a scatter plot.

This question studies the relationship between **Popularity and Duration**. We can create a scatter plot with duration on the X-axis and popularity on the Y-axis using `plt.scatter(df['Duration (ms)'], df['Popularity'])`. Each point represents one track. If the points show a clear upward pattern, there may be a positive relationship; if they show a downward pattern, there may be a negative relationship; and if they are randomly scattered, there may be little linear relationship. A scatter plot is useful because it allows us to visually inspect two numerical variables together.

4. Which artist has the highest number of tracks in the dataset? Display the count of tracks for each artist using a countplot.

This question asks us to find which artist appears most frequently in the dataset. We can count the number of tracks associated with each artist using `df['Artist'].value_counts()`. A **countplot** can then be used to display these counts visually. The artist with the tallest bar has the highest number of tracks in the dataset.

This helps us understand which artists are most represented in the collected Spotify data.

5. What are the top 5 least popular tracks in the dataset? Provide the artist name and track name for each.

The Popularity column contains a numerical popularity score for each track. To find the five least popular tracks, we sort the DataFrame by Popularity in ascending order and select the first five records. We then display the Artist and Track Name columns for those records. This directly identifies the five tracks with the lowest popularity scores in the dataset.

6. Among the top 5 most popular artists, which artist has the highest popularity on average? Calculate and display the average popularity for each artist.

First, we need to determine the **top five artists according to a defined measure of popularity**. If the intention is based on the number of popular tracks represented in the dataset, we can identify the five artists with the strongest representation and then calculate each artist's average track popularity. We can use `groupby('Artist')['Popularity'].mean()` to calculate average popularity. We then compare these averages and identify which artist has the highest mean popularity. The important point is that "top 5 most popular artists" should be defined using a measurable criterion before performing the calculation.

7. For the top 5 most popular artists, what are their most popular tracks? List the track name for each artist.

After identifying the five artists according to the chosen popularity criterion, we can find the track with the highest Popularity value for each of those artists. One way is to group the records by artist and select the row with the maximum popularity for each group. The result gives us the most popular track associated with each selected artist. This combines **grouping, filtering, and finding maximum values**.

8. Visualize relationships between multiple numerical variables simultaneously using a pair plot.

A **pair plot** is a visualization that displays relationships between multiple numerical variables at the same time. For the Spotify dataset, numerical variables include Popularity and Duration (ms), while other columns such as Artist and Track Name are categorical/text values. We can use Seaborn's `pairplot()` on the relevant numerical columns. The diagonal usually shows the distribution of each variable, while the other plots show relationships between pairs of numerical variables. This is useful during EDA because it gives us a quick overall view of numerical relationships.

9. Does the duration of tracks vary significantly across different artists? Explore this visually using a box plot or violin plot.

This question compares **track duration among different artists**. A box plot can show the median, quartiles, spread, and potential outliers of track duration for each artist. A violin plot additionally shows the shape of the distribution. If some artists have much wider or different distributions of duration than others, it suggests that their track lengths vary differently. We should be careful with the word "significantly": a visual plot can show noticeable differences, but statistical significance would normally require an appropriate statistical test.

10. How does the distribution of track popularity vary for different artists? Visualize this using a swarm plot or a violin plot.

Here we compare the **Popularity distribution across artists**. A violin plot can show the shape and spread of popularity values for each artist, while a swarm plot displays individual observations. By comparing these plots, we can see which artists have generally higher popularity, how widely their popularity scores vary, and whether there are unusual tracks. This is another example of comparing one numerical variable across different categories.